

Unit 3.3: The Inverse Transformation Method

Dr. Manoj C. Patil

2026-09-04

Learning Objectives

By the end of this section, a student should be able to

- explain why simulation of an arbitrary random variable can be reduced to simulation of a single $\text{Uniform}(0, 1)$ random variable,
- state and prove the probability integral transformation theorem,
- define the generalized inverse (quantile function) of a cumulative distribution function and describe its essential properties,
- derive the distribution of the quantile function evaluated at a $\text{Uniform}(0, 1)$ random variable,
- construct the inverse transformation algorithm for continuous random variables and implement it in R,
- construct the inverse transformation algorithm for discrete random variables and implement it in R,
- assess the computational advantages and limitations of the method relative to alternative generation techniques.

Prerequisites

A reader should already be comfortable with

- the definition and basic properties of a cumulative distribution function (cdf), including right-continuity and monotonicity,
- the notions of a continuous and a discrete random variable,
- elementary calculus, including differentiation and integration of standard functions,
- the `runif()` function in R and vectorized R operations.

Introduction

Every simulation of a stochastic system eventually reduces to a very small number of primitive operations, the most fundamental of which is the generation of a variate that behaves, for practical purposes, like a draw from the $\text{Uniform}(0, 1)$ distribution. Unit 3.1 discussed how such uniform variates are produced algorithmically using congruential generators. The question addressed in this section is the converse one: given a stream of $\text{Uniform}(0, 1)$ numbers, how does one convert it into a stream of numbers that follow a prescribed target distribution, such as an Exponential, a Gamma, or a Binomial distribution?

The inverse transformation method answers this question in essentially the most direct way possible. It exploits the fact that every random variable can be described completely by its cumulative distribution function, and that this function itself carries enough information to be inverted and used as a generating mechanism. The idea has a long history in statistics: it underlies the classical technique of generating an observation by first drawing a random probability and then reading off the corresponding quantile, a procedure that predates electronic computing and appears in early Monte Carlo work carried out by physicists such as Ulam and von Neumann in the 1940s in connection with the Manhattan Project. Later authors, notably Fishman (1996) and Devroye (1986), placed the method on a rigorous footing and catalogued its use across a wide range of standard distributions.

The practical importance of the method lies in its generality and its conceptual simplicity. Whenever the inverse cdf can be written down in closed form, or computed cheaply by numerical inversion, the method provides an exact, unbiased, and easily coded generator. It is for this reason the default strategy taught first in any course on stochastic simulation, and it forms the conceptual foundation against which more elaborate techniques, such as acceptance-rejection and composition methods, are later compared.

Intuition

Consider a continuous random variable X with a strictly increasing cumulative distribution function F_X . Because F_X is strictly increasing, it maps the real line onto the interval $(0, 1)$ in a one-to-one fashion, and the value $F_X(x)$ can be read as “the proportion of the distribution lying at or below x .” This proportion is itself a number between 0 and 1, and a remarkable fact is that if X is random with distribution F_X , then the transformed quantity $F_X(X)$ is random with the Uniform(0, 1) distribution, regardless of what F_X actually is. Intuitively, F_X stretches and compresses different regions of the real line so that, after the transformation, every sub-interval of $(0, 1)$ of a given length receives exactly the same amount of probability mass.

This observation can be turned around. If stretching X through F_X produces a uniform variate, then the reverse operation — starting from a uniform variate U and applying the inverse function F_X^{-1} — must produce a variate distributed as X . Generating X therefore reduces to two steps: draw U uniformly on $(0, 1)$, and report $x = F_X^{-1}(u)$ as the simulated value. A useful mental image is that of reading a vertical line at height u across the graph of F_X until it meets the curve, and then dropping down to the corresponding value on the horizontal axis; that horizontal value is the simulated observation.

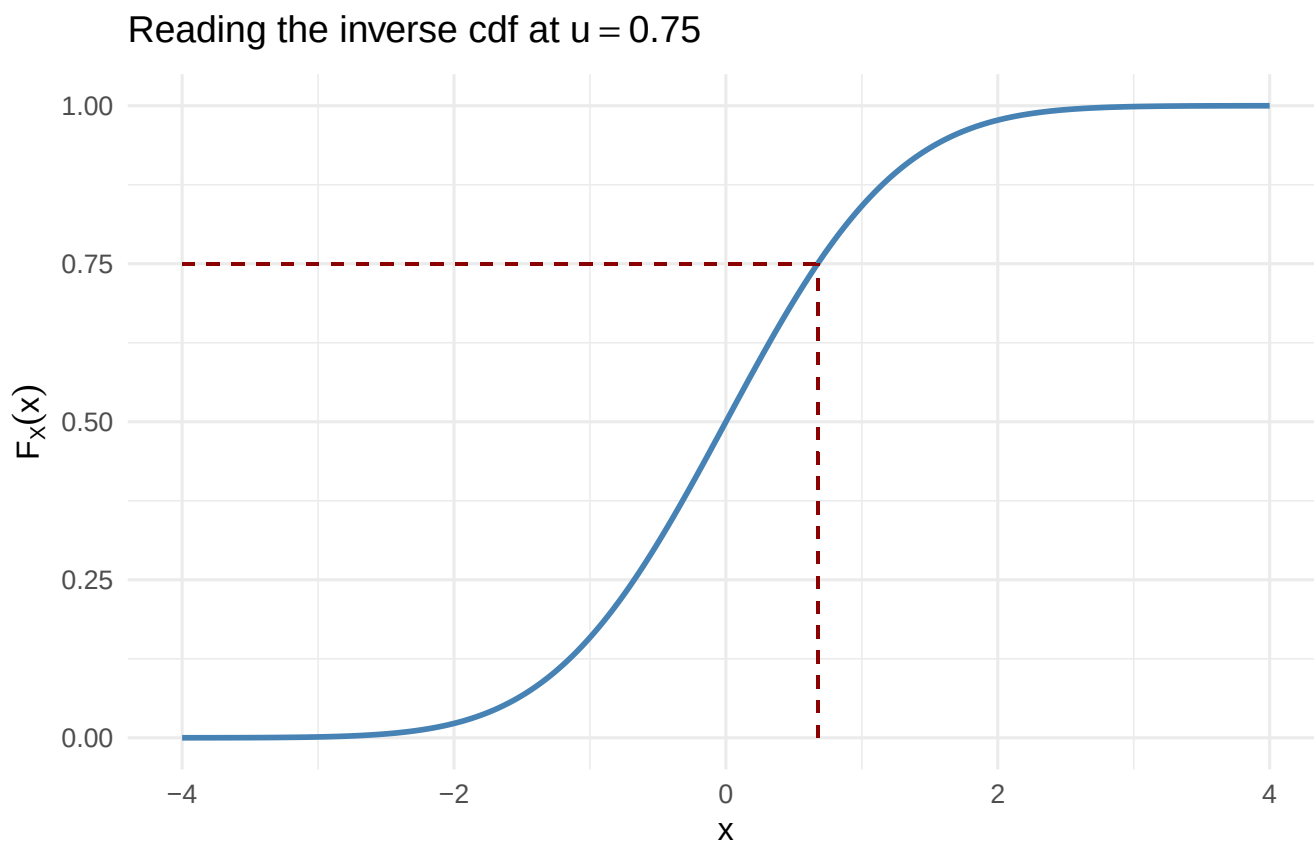


Figure 1: Geometric idea of the inverse transformation: a draw u on the vertical axis is mapped, through the cdf, to the simulated value x on the horizontal axis.

The situation for a discrete random variable is only slightly more delicate, because F_X is then a step function rather than a smooth curve, so its inverse cannot be defined by simple algebraic inversion. The remedy is to define the inverse in a slightly generalized sense, described formally below, which continues to make sense whether F_X is continuous, discrete, or a mixture of the two.

Formal Definitions

The Quantile Function

Definition. For a cumulative distribution function F_X , the **quantile function**, or **generalized inverse**, is defined for $0 < u < 1$ by

$$F_X^{-1}(u) = \inf\{x \in \mathbb{R} : F_X(x) \geq u\}.$$

where

- u denotes a probability level in $(0, 1)$,
- $F_X(x)$ denotes the cumulative distribution function of X evaluated at x ,
- $\inf$ denotes the infimum, that is, the greatest lower bound of the indicated set.

When F_X is continuous and strictly increasing, F_X^{-1} coincides with the ordinary functional inverse of F_X , so that $F_X(F_X^{-1}(u)) = u$ for every $u \in (0, 1)$. When F_X has jumps, as it does for a discrete random variable, or flat segments, the generalized definition still produces a single well-defined value $F_X^{-1}(u)$ for every u , namely the smallest x at which the cdf reaches or exceeds the level u .

Properties of the Quantile Function

The quantile function possesses three properties that make it suitable as a generating device.

- **Property 1 (Non-decreasing).** F_X^{-1} is a non-decreasing function of u on $(0, 1)$, because F_X itself is non-decreasing.
- **Property 2 (Left-continuity).** F_X^{-1} is left-continuous, a mirror image of the right-continuity of F_X .
- **Property 3 (Inversion identity).** For every $x \in \mathbb{R}$ and every $u \in (0, 1)$,

$$F_X^{-1}(u) \leq x \iff u \leq F_X(x).$$

This equivalence, sometimes called the **compatibility relation**, is the identity on which the probability integral transformation theorem is built, and it holds regardless of whether F_X is continuous or discrete.

Theorems

Theorem 3.1: Probability Integral Transformation

Statement. Let X be a continuous random variable with cdf F_X . Then the random variable

$$U = F_X(X)$$

follows the Uniform(0, 1) distribution.

Assumptions.

- F_X is continuous on $\mathbb{R}$.
- F_X takes values only in $(0, 1)$ over the support of X (the boundary values 0 and 1 occur with probability zero for a continuous distribution).

Proof. Let $u \in (0, 1)$. Using the compatibility relation from Property 3,

$$\begin{aligned} P(U \leq u) &= P(F_X(X) \leq u) \\ &= P(X \leq F_X^{-1}(u)) \quad (\text{by continuity of } F_X, \text{ equality holds a.s.}) \\ &= F_X(F_X^{-1}(u)) \\ &= u. \end{aligned}$$

The final equality uses the fact that, for a continuous cdf, applying F_X to its own generalized inverse returns the original argument. Since $P(U \leq u) = u$ for every $u \in (0, 1)$, U has the Uniform(0, 1) cdf, and the theorem follows. ■

Remarks. The theorem is often summarized by saying that “the cdf of a continuous random variable, evaluated at that random variable, is uniform.” It is the theoretical engine behind numerous statistical procedures beyond simulation, including goodness-of-fit testing (the Kolmogorov–Smirnov test of Section 3.2 relies on exactly this fact) and copula theory.

Examples. If $X \sim \text{Exponential}(\lambda)$ with $F_X(x) = 1 - e^{-\lambda x}$ for $x > 0$, then $U = 1 - e^{-\lambda X}$ is Uniform(0, 1); this can be verified numerically by simulating a large exponential sample and applying the transformation.

Applications. Probability integral transformation is the basis of copula-based dependence modelling, quantile-quantile plotting for distributional diagnostics, and simulation-based p-value calibration.

Theorem 3.2: The Inverse Transformation Theorem

Statement. Let $U \sim \text{Uniform}(0, 1)$, and let F_X be any cumulative distribution function with generalized inverse F_X^{-1} . Then the random variable

$$X = F_X^{-1}(U)$$

has cumulative distribution function F_X .

Assumptions.

- F_X is a valid cdf (non-decreasing, right-continuous, with limits 0 and 1 at $-\infty$ and $+\infty$ respectively).
- No continuity or strict monotonicity of F_X is required; the theorem holds for continuous, discrete, and mixed distributions alike.

Proof. Fix $x \in \mathbb{R}$. Using the compatibility relation of Property 3,

$$\begin{aligned} P(X \leq x) &= P(F_X^{-1}(U) \leq x) \\ &= P(U \leq F_X(x)) \\ &= F_U(F_X(x)) \\ &= F_X(x), \end{aligned}$$

where the third equality uses the Uniform(0, 1) cdf $F_U(t) = t$ for $t \in (0, 1)$, and $F_X(x) \in [0, 1]$ automatically. Since $P(X \leq x) = F_X(x)$ for every x , the random variable $X = F_X^{-1}(U)$ has the required distribution. ■

Remarks. This theorem, rather than Theorem 3.1, is the one actually used for generation: it converts a single recipe — draw U , evaluate F_X^{-1} — into a variate from any target distribution, continuous or discrete, as long as F_X^{-1} can be evaluated. The two theorems are converses of one another, and together they establish a full equivalence between “being distributed as X ” and “being obtainable from a uniform variate via F_X^{-1} .”

Examples. These are developed in full in the worked examples below for both the continuous and the discrete case.

Applications. The inverse transformation theorem underlies random variate generation in nearly every general-purpose statistical package, is used to couple correlated simulations sharing common random numbers (a variance-reduction technique), and provides the generating mechanism for parametric bootstrap resampling from a fitted distribution.

Mathematical Derivations

Distribution of the Quantile Function at a Uniform Argument

It is instructive to see explicitly why the derivation above does not depend on continuity. Write $G(u) = F_X^{-1}(u)$ for brevity, and recall the compatibility relation

$$G(u) \leq x \iff u \leq F_X(x), \quad x \in \mathbb{R}, u \in (0, 1).$$

For a fixed x , define $u_0 = F_X(x)$. The relation states that the event $\{G(U) \leq x\}$ and the event $\{U \leq u_0\}$ are the same event, as sets of outcomes, whatever the shape of F_X . Since U is $\text{Uniform}(0, 1)$,

$$P(U \leq u_0) = u_0 = F_X(x),$$

and therefore $P(G(U) \leq x) = F_X(x)$ for every real x , which is precisely the statement that $G(U) = F_X^{-1}(U)$ has cdf F_X . The argument used only the compatibility relation, so it is equally valid when F_X has jumps (the discrete case) or flat stretches (a distribution with an atom followed by a continuous part). This is the reason the inverse transformation method extends, without modification in principle, from continuous to discrete random variables; only the practical evaluation of F_X^{-1} changes.

Continuous Case: Explicit Inversion

When F_X is continuous and strictly increasing on the support of X , F_X^{-1} can, in many standard families, be written down by solving $F_X(x) = u$ algebraically for x .

Derivation for the Exponential distribution. Let $X \sim \text{Exponential}(\lambda)$, so that $F_X(x) = 1 - e^{-\lambda x}$ for $x > 0$. Setting $F_X(x) = u$,

$$\begin{aligned} 1 - e^{-\lambda x} &= u \\ e^{-\lambda x} &= 1 - u \\ -\lambda x &= \log(1 - u) \\ x &= -\frac{1}{\lambda} \log(1 - u). \end{aligned}$$

Hence $F_X^{-1}(u) = -\lambda^{-1} \log(1 - u)$. Because U and $1 - U$ have the same $\text{Uniform}(0, 1)$ distribution, it is equally valid, and slightly more economical in code, to generate

$$X = -\frac{1}{\lambda} \log(U).$$

Derivation for the density $f(x) = 3x^2$, $0 < x < 1$. Integrating, $F_X(x) = x^3$ for $0 < x < 1$. Setting $x^3 = u$ gives

$$F_X^{-1}(u) = u^{1/3}.$$

Discrete Case: Inversion by Search

When X is discrete with possible values $x_1 < x_2 < x_3 < \dots$ at which F_X jumps, the compatibility relation specializes to

$$F_X^{-1}(u) = x_i \quad \text{where} \quad F_X(x_{i-1}) < u \leq F_X(x_i).$$

Generating X therefore requires locating the first value x_i at which the cumulative sum of probabilities reaches or exceeds u . Unlike the continuous case, there is generally no closed-form algebraic shortcut, and the inversion is instead carried out by searching a table of cumulative probabilities, either sequentially or, for large tables, using a fast lookup such as a binary search.

Derivation for a two-point (Bernoulli) distribution. Let $X \sim \text{Bernoulli}(p)$, so $F_X(0) = 1 - p$ and $F_X(1) = 1$. Then

$$F_X^{-1}(u) = \begin{cases} 0, & 0 < u \leq 1 - p, \\ 1, & 1 - p < u < 1. \end{cases}$$

Derivation for a geometric distribution. Let $X \sim \text{Geometric}(p)$ on $\{0, 1, 2, \dots\}$ with $q = 1 - p$ and cdf $F_X(x) = 1 - q^{x+1}$. Solving $F_X(x - 1) < u \leq F_X(x)$,

$$1 - q^x < u \leq 1 - q^{x+1} \quad \Longleftrightarrow \quad x < \frac{\log(1 - u)}{\log q} \leq x + 1,$$

so that $x + 1 = \lceil \log(1 - u) / \log q \rceil$, where $\lceil t \rceil$ denotes the ceiling function (the smallest integer not less than t). Because U and $1 - U$ are identically distributed, this simplifies in implementation to

$$X = \left\lfloor \frac{\log U}{\log q} \right\rfloor.$$

Algorithms

Algorithm 3.1: Inverse Transformation Method, Continuous Case

Purpose. Generate a random sample from a continuous distribution whose quantile function F_X^{-1} is available in closed form or can be evaluated numerically.

Input. Sample size n ; the quantile function $F_X^{-1}(\cdot)$ (or the parameters needed to evaluate it).

Output. A vector $x_1, \dots, x_n$ of simulated observations from the target distribution.

Step-by-step procedure.

1. Derive the quantile function $F_X^{-1}(u)$ analytically by solving $F_X(x) = u$ for x .
2. Implement $F_X^{-1}(u)$ as an R expression or function.
3. For $i = 1, \dots, n$:
 - (a) draw $u_i \sim \text{Uniform}(0, 1)$;
 - (b) compute $x_i = F_X^{-1}(u_i)$.
4. Return the vector $(x_1, \dots, x_n)$.

Pseudo-code.

Algorithm: Inverse Transform (Continuous)

Input: n , quantile function `Finv`

Output: vector `x` of length n

```

1. Generate  $u[1], \dots, u[n]$  each  $\text{Uniform}(0,1)$ 
2. For  $i = 1$  to  $n$ :
     $x[i] \leftarrow \text{Finv}(u[i])$ 
3. Return  $x$ 

```

Flowchart description. Start, draw a uniform vector of length n , apply the (vectorized) quantile function element-wise, obtain the simulated sample, end.

Complexity. Generating n uniform variates costs $O(n)$, and evaluating a closed-form quantile function at n points also costs $O(n)$ under vectorization, so the overall time complexity is $O(n)$. Memory usage is $O(n)$ for storage of the sample.

Advantages. Extremely fast when a closed form is available; requires only one uniform draw per observation; naturally vectorized in R.

Limitations. Requires an explicit or numerically invertible quantile function; some distributions (e.g. the standard Normal) have no elementary closed form, and rely on numerical approximations implemented in built-in functions such as `qnorm()`.

Applications. Simulating waiting times (Exponential), simulating order statistics via the Beta quantile function, generating covariates in Monte Carlo studies, and parametric bootstrap resampling.

Algorithm 3.2: Inverse Transformation Method, Discrete Case

Purpose. Generate a random sample from a discrete distribution supported on a countable set of values, by searching a table of cumulative probabilities.

Input. Sample size n ; the support points $x_1 < x_2 < \dots < x_k$; the corresponding cumulative probabilities $F_X(x_1), \dots, F_X(x_k)$.

Output. A vector $x_1^*, \dots, x_n^*$ of simulated observations.

Step-by-step procedure.

1. Compute (or look up) the cumulative probability vector $F = (F_X(x_1), \dots, F_X(x_k))$.
2. For $i = 1, \dots, n$:
 - (a) draw $u_i \sim \text{Uniform}(0, 1)$;
 - (b) find the smallest index j such that $F_X(x_j) \geq u_i$;
 - (c) set $x_i^* = x_j$.
3. Return the vector $(x_1^*, \dots, x_n^*)$.

Pseudo-code.

Algorithm: Inverse Transform (Discrete)

Input: n , support $(x_1, \dots, x_k)$, cdf values $(F_1, \dots, F_k)$

Output: vector x of length n

```

1. Generate  $u[1], \dots, u[n]$  each  $\text{Uniform}(0,1)$ 
2. For  $i = 1$  to  $n$ :
     $j \leftarrow$  smallest index such that  $F[j] \geq u[i]$ 
     $x[i] \leftarrow \text{support}[j]$ 
3. Return  $x$ 

```

Flowchart description. Start, build or retrieve the cumulative probability table, draw a uniform vector, for each uniform value scan (or binary-search) the table for the first cumulative probability at or above it, record the corresponding support point, end.

Complexity. With linear search of a table of size k , generating n observations costs $O(nk)$ in the worst case. Using a pre-sorted table with binary search reduces this to $O(n \log k)$. Vectorized R implementations exploiting `findInterval()` achieve the $O(n \log k)$ bound efficiently in practice.

Advantages. Works for any discrete distribution with a computable cdf, including distributions without closed-form quantile expressions; straightforward to implement using base R search utilities.

Limitations. Slower than direct continuous inversion because of the search step; for distributions with unbounded support (e.g. the Poisson or the logarithmic distribution used later in this unit) the cumulative table must be truncated or extended adaptively.

Applications. Simulating categorical survey responses, generating Bernoulli and Binomial trials, bootstrap resampling from an empirical distribution, simulating claim counts in actuarial risk models.

R Programming

The following code implements both algorithms. Each example follows the pattern: state the target distribution, derive or specify the quantile function, generate the sample, and compare it graphically and numerically against the theoretical distribution.

Continuous Case: Density $f(x) = 3x^2$, $0 < x < 1$

```
set.seed(123)

## Target density f(x) = 3x^2 on (0,1) has cdf F(x) = x^3
## so the quantile function is F^{-1}(u) = u^{1/3}

sample_size <- 1000
uniform_draws <- runif(n = sample_size)
simulated_x <- uniform_draws^(1 / 3)

summary(simulated_x)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.07749 0.63300 0.78835 0.74867 0.90727 0.99980

theoretical_grid <- seq(0, 1, by = 0.01)
theoretical_density <- 3 * theoretical_grid^2

ggplot(data.frame(x = simulated_x), aes(x = x)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30, fill = "steelblue", colour = "white", alpha = 0.8
  ) +
  geom_line(
    data = data.frame(x = theoretical_grid, y = theoretical_density),
    aes(x = x, y = y),
    colour = "darkred", linewidth = 1
  ) +
  labs(
    title = expression(paste("Simulated vs. theoretical density, ", f(x) == 3 * x^2)),
    x = "x", y = "Density"
  ) +
  theme_minimal(base_size = 12)
```

Continuous Case: Exponential Distribution

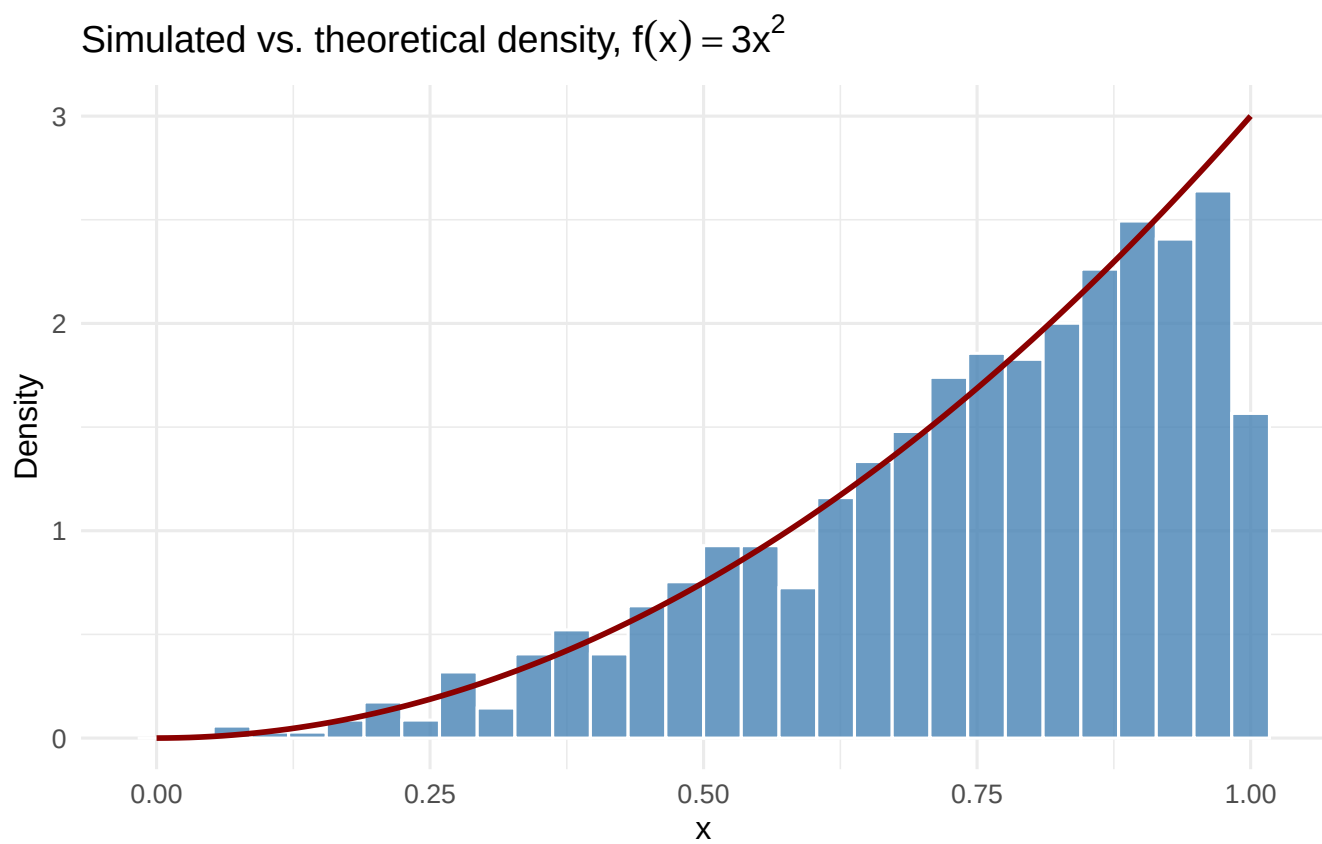


Figure 2: Simulated density (histogram) versus the theoretical density $f(x) = 3x^2$ superimposed as a curve.

```

set.seed(123)

##  $X \sim \text{Exponential}(\text{rate} = \lambda)$ ;  $F^{-1}(u) = -\log(u) / \lambda$ 
sample_size <- 1000
lambda <- 2

uniform_draws <- runif(n = sample_size)
simulated_exp <- -log(uniform_draws) / lambda

sample_mean <- mean(simulated_exp)
theoretical_mean <- 1 / lambda

kable(
  data.frame(
    Quantity = c("Sample mean", "Theoretical mean (1/lambda)"),
    Value = round(c(sample_mean, theoretical_mean), 4)
  ),
  caption = "Comparison of sample and theoretical mean, Exponential(2)"
) %>%
  kable_styling(full_width = FALSE)

```

Table 1: Comparison of sample and theoretical mean, Exponential(2)

Quantity	Value
Sample mean	0.5032
Theoretical mean (1/lambda)	0.5000

Discrete Case: Bernoulli Distribution

```

set.seed(123)

##  $X \sim \text{Bernoulli}(p)$ ;  $F(0) = 1-p$ ,  $F(1) = 1$ 
sample_size <- 1000
success_prob <- 0.4

uniform_draws <- runif(n = sample_size)
simulated_bernoulli <- as.integer(uniform_draws > (1 - success_prob))

kable(
  data.frame(
    Quantity = c("Sample mean", "Theoretical mean (p)",
                 "Sample variance", "Theoretical variance p(1-p)"),
    Value = round(
      c(mean(simulated_bernoulli), success_prob,
        var(simulated_bernoulli), success_prob * (1 - success_prob)),
      4
    )
  ),
  caption = "Comparison of sample and theoretical moments, Bernoulli(0.4)"
) %>%
  kable_styling(full_width = FALSE)

```

Table 2: Comparison of sample and theoretical moments, Bernoulli(0.4)

Quantity	Value
Sample mean	0.3930
Theoretical mean (p)	0.4000
Sample variance	0.2388
Theoretical variance $p(1-p)$	0.2400

Discrete Case: Geometric Distribution

```
set.seed(123)

## X ~ Geometric(p) on {0,1,2,...}; using X = floor(log(u) / log(1-p))
sample_size <- 1000
success_prob <- 0.25

uniform_draws <- runif(n = sample_size)
simulated_geometric <- floor(log(uniform_draws) / log(1 - success_prob))

sample_mean <- mean(simulated_geometric)
theoretical_mean <- (1 - success_prob) / success_prob

kable(
  data.frame(
    Quantity = c("Sample mean", "Theoretical mean ((1-p)/p)"),
    Value = round(c(sample_mean, theoretical_mean), 4)
  ),
  caption = "Comparison of sample and theoretical mean, Geometric(0.25)"
) %>%
  kable_styling(full_width = FALSE)
```

Table 3: Comparison of sample and theoretical mean, Geometric(0.25)

Quantity	Value
Sample mean	3.03
Theoretical mean $((1-p)/p)$	3.00

Discrete Case: General Table Lookup with findInterval()

```
set.seed(123)

## General discrete distribution on support {1,2,3,4} with given pmf
support_points <- c(1, 2, 3, 4)
pmf_values <- c(0.1, 0.2, 0.3, 0.4)
cdf_values <- cumsum(pmf_values)

sample_size <- 1000
uniform_draws <- runif(n = sample_size)
```

```
## findInterval locates, for each u, the index of the first cdf value >= u
indices <- findInterval(
  x = uniform_draws,
  vec = cdf_values,
  left.open = TRUE
) + 1
simulated_general <- support_points[indices]

table(simulated_general) / sample_size

## simulated_general
##      1      2      3      4
## 0.099 0.197 0.311 0.393
```

Simulation Studies

Purpose

The purpose of this simulation study is to verify, for both a continuous and a discrete example, that the empirical distribution produced by the inverse transformation algorithm converges to the theoretical target distribution as the sample size increases, and to illustrate the rate of that convergence.

Code

```
set.seed(123)

sample_sizes <- c(50, 200, 1000, 5000)
lambda <- 1

convergence_results <- map_dfr(sample_sizes, function(n) {
  u <- runif(n)
  x <- -log(u) / lambda
  tibble(
    n = n,
    sample_mean = mean(x),
    sample_sd = sd(x)
  )
})

kable(
  convergence_results,
  col.names = c("Sample size (n)", "Sample mean", "Sample SD"),
  caption = "Convergence of sample moments to theoretical values (mean = 1, sd = 1) as n increases"
) %>%
  kable_styling(full_width = FALSE)
```

Table 4: Convergence of sample moments to theoretical values
(mean = 1, sd = 1) as n increases

Sample size (n)	Sample mean	Sample SD
50	0.9104796	0.8633936
200	0.9138954	0.8678713

1000	1.0297235	1.0233799
5000	1.0005627	0.9987856

Output

```
set.seed(123)

simulated_data <- map_dfr(sample_sizes, function(n) {
  u <- runif(n)
  x <- -log(u) / lambda
  tibble(n = factor(n), x = x)
})

ggplot(simulated_data, aes(x = x, colour = n)) +
  geom_density(linewidth = 0.9) +
  stat_function(
    fun = dexp, args = list(rate = lambda),
    colour = "black", linetype = "dashed", linewidth = 1
  ) +
  labs(
    title = "Convergence of simulated density to the Exponential(1) target",
    x = "x", y = "Density", colour = "Sample size"
  ) +
  theme_minimal(base_size = 12)
```

Interpretation and Discussion

The table shows that the sample mean and sample standard deviation approach the theoretical values of 1 and 1 (for a Rate= 1 Exponential distribution) as n grows, with the fluctuation around these targets shrinking approximately in proportion to $1/\sqrt{n}$, consistent with the law of large numbers and the central limit theorem applied to the sample mean. The density plot exhibits the same pattern visually: the estimated curve for $n = 50$ is jagged and departs noticeably from the theoretical dashed curve, whereas the curve for $n = 5000$ is smooth and tracks the theoretical density closely. This confirms that the inverse transformation algorithm is not merely producing values with the correct support, but is reproducing the full shape of the target distribution, as guaranteed by Theorem 3.2. The exercise also illustrates a general principle in simulation practice: convergence diagnostics of this kind should always accompany a newly implemented generator before it is trusted in downstream analyses.

Graphs

The figures embedded in the R Programming and Simulation Studies sections above were produced entirely in R using `ggplot2`, following the house style of this course. Each figure carries a title, axis labels, and, where more than one series is displayed, a legend distinguishing the series (see the convergence plot, where colour encodes sample size).

Numerical Examples

Example 1: Density $f(x) = 3x^2$

Problem. Using the inverse transformation method, describe how a single observation is generated from the density $f(x) = 3x^2$, $0 < x < 1$, given the uniform draw $u = 0.729$.

Given. $F_X(x) = x^3$, hence $F_X^{-1}(u) = u^{1/3}$; $u = 0.729$.

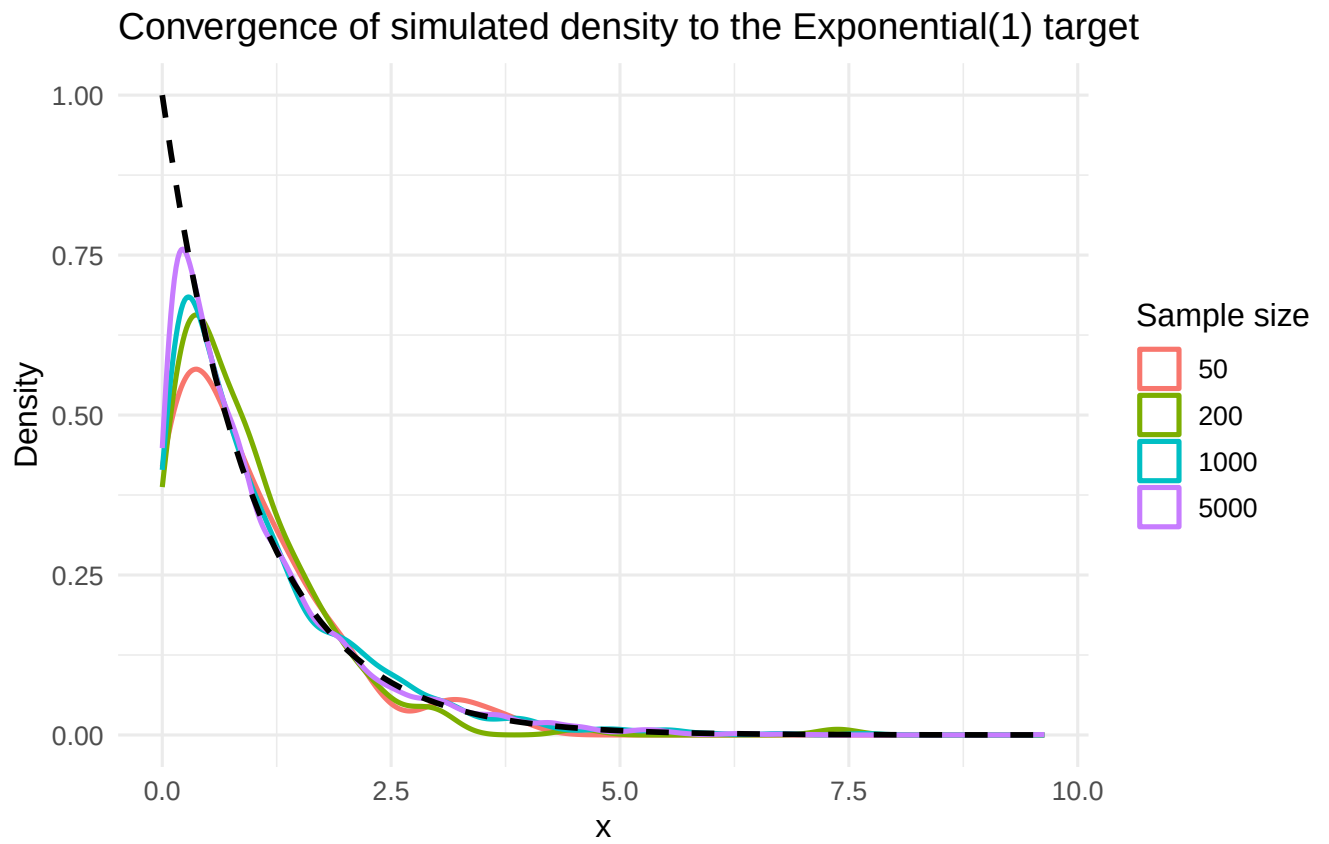


Figure 3: Kernel density estimates of simulated Exponential(1) samples at increasing sample sizes, compared with the theoretical density.

Solution.

$$x = u^{1/3} = 0.729^{1/3} = 0.9.$$

Interpretation. The value $u = 0.729$, interpreted as a cumulative probability, corresponds to the point $x = 0.9$ under this distribution, meaning that 72.9% of the probability mass of X lies at or below 0.9, consistent with $F_X(0.9) = 0.9^3 = 0.729$.

Example 2: Exponential Distribution

Problem. Using the inverse transformation method with rate $\lambda = 0.5$, find the simulated observation corresponding to the uniform draw $u = 0.2$.

Given. $F_X^{-1}(u) = -\log(u)/\lambda$; $\lambda = 0.5$; $u = 0.2$.

Solution.

$$x = -\frac{\log(0.2)}{0.5} = -\frac{-1.6094}{0.5} = 3.2189.$$

Interpretation. A relatively small uniform draw of 0.2 produces a moderately large exponential observation, illustrating that small values of u near zero are mapped to the lower tail of F_X , which for the exponential distribution corresponds to values further into the right tail because F_X rises slowly for small x relative to $\lambda = 0.5$.

Example 3: Bernoulli Distribution

Problem. With success probability $p = 0.4$, determine the simulated value of $X \sim \text{Bernoulli}(0.4)$ corresponding to uniform draws $u = 0.55$ and $u = 0.70$.

Given. $F_X(0) = 0.6$, $F_X(1) = 1$.

Solution. For $u = 0.55$: since $0.55 \leq 0.6 = F_X(0)$, the generated value is $x = 0$. For $u = 0.70$: since $0.70 > 0.6$, the generated value is $x = 1$.

Interpretation. Uniform draws at or below $F_X(0) = 1 - p = 0.6$ are mapped to the outcome 0, and draws above that threshold are mapped to 1; the length of each sub-interval of $(0, 1)$ mapped to a given outcome equals the probability of that outcome, here 0.6 for outcome 0 and 0.4 for outcome 1.

Example 4: Geometric Distribution

Problem. With $p = 0.25$, determine the simulated value of $X \sim \text{Geometric}(0.25)$ on $\{0, 1, 2, \dots\}$ corresponding to the uniform draw $u = 0.6$.

Given. $q = 1 - p = 0.75$; $X = \lfloor \log(u)/\log(q) \rfloor$.

Solution.

$$x = \left\lfloor \frac{\log(0.6)}{\log(0.75)} \right\rfloor = \left\lfloor \frac{-0.5108}{-0.2877} \right\rfloor = \lfloor 1.7756 \rfloor = 1.$$

Interpretation. The generated value $x = 1$ means that, under this draw, the process required one “failure” before the first “success” in the underlying Bernoulli trial sequence interpretation of the geometric distribution.

Applications

Statistics. The inverse transformation method provides the standard mechanism for parametric bootstrap resampling, in which observations are drawn from a fitted parametric cdf rather than from the empirical distribution.

Machine Learning. Data augmentation pipelines that need to inject synthetic covariates from a specified marginal distribution frequently rely on inverse transformation, particularly within copula-based generative models.

Data Science. Exploratory simulation of “what-if” scenarios, such as generating synthetic customer arrival times from a fitted distribution, uses the method directly.

Finance. Simulation of asset return distributions, value-at-risk estimation via Monte Carlo, and simulation of interarrival times of trades in market microstructure models all employ inverse transformation for the marginal distributions involved.

Reliability. Failure-time simulation from Exponential, Weibull, or Gamma lifetime models is a direct application, since these distributions admit closed-form or easily computed quantile functions.

Monte Carlo. The method is the workhorse for generating the primitive random inputs of a Monte Carlo integration or estimation scheme whenever the required marginal distribution is available in closed form.

Bayesian Statistics. Inverse transformation is used to sample from full conditional distributions in Gibbs sampling steps whenever those conditionals belong to a standard family with a tractable quantile function.

Operations Research. Discrete-event simulation of queueing systems relies on the discrete and continuous inverse transformation algorithms to generate service times, interarrival times, and categorical routing decisions.

Engineering. Simulation of component lifetimes and stress-test scenarios in reliability engineering commonly draws on the continuous inverse transformation algorithm applied to lifetime distributions.

Advantages

- Conceptually simple and directly justified by Theorem 3.2.
- Requires exactly one uniform draw per simulated observation, which is efficient when uniform generation is the bottleneck.
- Exact: the simulated variate has precisely the target distribution, with no approximation error beyond floating-point rounding.
- Naturally vectorizable in R, giving fast implementations for large sample sizes.
- Extends without conceptual change from continuous to discrete distributions via the generalized inverse.

Limitations

- Requires an explicit or efficiently computable quantile function; some important distributions, such as the standard Normal, have no elementary closed-form inverse and must rely on numerical approximation algorithms internally (as implemented in `qnorm()`).
- For discrete distributions with large or unbounded support, table search adds computational overhead relative to specialized generators.
- Distributions defined only implicitly, for example through a density known up to a normalizing constant, are typically not amenable to this method and require alternatives such as acceptance-rejection (Unit 3.4).

Practical Tips

- Always verify that the derived quantile function is applied to the correct branch of the cdf; sign errors in solving $F_X(x) = u$ are a common source of bugs.
- Exploit the identity that U and $1 - U$ share the same $\text{Uniform}(0, 1)$ distribution to simplify formulas, as done for the Exponential and Geometric cases above.
- When implementing the discrete case, prefer `findInterval()` over an explicit loop for speed; a naive `for` loop searching a cumulative table becomes slow for large sample sizes.
- Always set a seed with `set.seed()` before a simulation intended to be reproducible, and document the seed value used.
- Check the boundary behaviour of a custom quantile function at $u \rightarrow 0^+$ and $u \rightarrow 1^-$ to ensure it does not produce `NaN` or `Inf` values inadvertently, particularly for distributions with unbounded support.

Common Mistakes

Students often confuse the **cdf** with the **quantile function**, attempting to plug a value of x into F_X^{-1} instead of a probability u . It is important to remember that F_X maps values of the random variable to probabilities, while F_X^{-1} maps probabilities back to values of the random variable; the two functions have different domains and ranges, and reversing them by mistake produces a nonsensical simulated sample. Another frequent error is neglecting the discrete case's need for a search step, and instead attempting to apply an algebraic inverse formula that does not exist because F_X is not continuous.

Summary

- Every continuous random variable X satisfies $F_X(X) \sim \text{Uniform}(0, 1)$ (Theorem 3.1), and conversely, $F_X^{-1}(U) \sim X$ for $U \sim \text{Uniform}(0, 1)$ (Theorem 3.2), where F_X^{-1} is the generalized inverse (quantile function) of F_X .
- The inverse transformation algorithm reduces simulation of any distribution with a computable quantile function to a single uniform draw followed by evaluation of F_X^{-1} .
- In the continuous case, F_X^{-1} is often available in closed form by algebraically solving $F_X(x) = u$.
- In the discrete case, $F_X^{-1}(u)$ is obtained by locating the smallest support point x_i for which $F_X(x_i) \geq u$, typically via table search.
- The method is exact and simple, but its efficiency and applicability depend on how cheaply F_X^{-1} can be evaluated.

Key Formulae

Concept | Formula |

||| | Quantile function | $F_X^{-1}(u) = \inf\{x : F_X(x) \geq u\}$ | | Compatibility relation | $F_X^{-1}(u) \leq x \iff u \leq F_X(x)$ | | Probability integral transform | $F_X(X) \sim \text{Uniform}(0, 1)$ | | Inverse transform generator | $X = F_X^{-1}(U)$, $U \sim \text{Uniform}(0, 1)$ | | Exponential quantile function | $F_X^{-1}(u) = -\lambda^{-1} \log(1 - u)$ | | Discrete inversion rule | $F_X^{-1}(u) = x_i$ where $F_X(x_{i-1}) < u \leq F_X(x_i)$ | | Geometric quantile function | $F_X^{-1}(u) = \lceil \log(1 - u) / \log(1 - p) \rceil - 1$ |

Key Terms

- **Cumulative distribution function (cdf):** the function $F_X(x) = P(X \leq x)$.
- **Quantile function (generalized inverse):** the function $F_X^{-1}(u) = \inf\{x : F_X(x) \geq u\}$.
- **Probability integral transformation:** the result that $F_X(X) \sim \text{Uniform}(0, 1)$ for continuous X .
- **Inverse transformation method:** a simulation technique that generates $X = F_X^{-1}(U)$ from a uniform variate U .

- **Support:** the set of values that a random variable can take with positive probability (discrete case) or positive density (continuous case).
- **Compatibility relation:** the equivalence $F_X^{-1}(u) \leq x \iff u \leq F_X(x)$.

Exercises

Multiple Choice Questions

- The inverse transformation method requires which of the following as its only source of randomness?
 - A Normal(0,1) variate
 - A Uniform(0,1) variate
 - A Poisson variate
 - A Bernoulli variate
- If X is continuous with cdf F_X , then $F_X(X)$ follows
 - the distribution of X
 - the Uniform(0,1) distribution
 - the standard Normal distribution
 - the Exponential(1) distribution
- The quantile function $F_X^{-1}(u)$ is defined as
 - $\sup\{x : F_X(x) \leq u\}$
 - $\inf\{x : F_X(x) \geq u\}$
 - the derivative of F_X
 - the reciprocal of F_X
- For $X \sim \text{Exponential}(\lambda)$, the quantile function is
 - $-\lambda \log(1 - u)$
 - $-\lambda^{-1} \log(1 - u)$
 - $\lambda^{-1} e^{-u}$
 - λu
- In the discrete case, $F_X^{-1}(u) = x_i$ whenever
 - $u = F_X(x_i)$ exactly
 - $F_X(x_{i-1}) < u \leq F_X(x_i)$
 - $F_X(x_i) < u \leq F_X(x_{i-1})$
 - $u < F_X(x_{i-1})$
- Which R function is most efficient for locating the correct discrete category during vectorized inverse transformation?
 - `sapply()` with a manual loop
 - `findInterval()`
 - `paste()`
 - `Recall()`
- Which identity is commonly exploited to simplify quantile function formulas?
 - U and U^2 are identically distributed
 - U and $1 - U$ are identically distributed
 - U and $\log U$ are identically distributed
 - U is always symmetric about 1
- The time complexity of continuous inverse transformation for n observations, assuming a closed-form quantile function, is
 - $O(n^2)$
 - $O(n \log n)$
 - $O(n)$
 - $O(\log n)$
- Which distribution does NOT have a simple closed-form quantile function?
 - Exponential
 - Uniform
 - standard Normal

- (d) two-point (Bernoulli)
10. The compatibility relation states that
- $F_X^{-1}(u) \leq x \iff u \leq F_X(x)$
 - $F_X^{-1}(u) \geq x \iff u \geq F_X(x)$
 - $F_X(u) \leq x \iff F_X^{-1}(x) \leq u$
 - F_X and F_X^{-1} are always equal

Answers. 1(b), 2(b), 3(b), 4(b), 5(b), 6(b), 7(b), 8(c), 9(c), 10(a).

Fill in the Blanks

- The inverse transformation method converts a _____ variate into a variate from any target distribution.
- The function $F_X^{-1}(u) = \inf\{x : F_X(x) \geq u\}$ is called the _____ function.
- The probability integral transformation states that $F_X(X)$ follows the _____ distribution.
- For $X \sim \text{Exponential}(\lambda)$, the quantile function is $F_X^{-1}(u) = -\lambda^{-1} \log(1-u)$.
- In the discrete case, $F_X^{-1}(u)$ is found by _____ a cumulative probability table.
- The R function _____ efficiently performs table lookup for vectorized discrete inversion.
- U and _____ share the same $\text{Uniform}(0,1)$ distribution.
- The quantile function is always _____ (increasing/decreasing) in u .
- Theorem 3.2 shows that $F_X^{-1}(U)$ has cdf equal to _____.
- The standard Normal distribution requires _____ inversion because no elementary closed form exists.

Answers. 1. uniform; 2. quantile (generalized inverse); 3. $\text{Uniform}(0,1)$; 4. $-\lambda^{-1} \log(1-u)$; 5. searching; 6. `findInterval()`; 7. $1-U$; 8. non-decreasing; 9. F_X ; 10. numerical.

True or False

- The inverse transformation method can be applied to discrete distributions. (True/False)
- $F_X(X)$ is $\text{Uniform}(0,1)$ only when X is discrete. (True/False)
- The quantile function is always continuous. (True/False)
- U and $1-U$ have the same distribution when $U \sim \text{Uniform}(0,1)$. (True/False)
- The exponential quantile function requires numerical root-finding. (True/False)
- The discrete inverse transformation algorithm never requires a search step. (True/False)
- Theorem 3.2 holds regardless of whether F_X is continuous. (True/False)
- `findInterval()` can be used to vectorize discrete inverse transformation. (True/False)
- The inverse transformation method always requires more than one uniform draw per observation. (True/False)
- The quantile function is non-decreasing in u . (True/False)

Answers. 1. True; 2. False; 3. False; 4. True; 5. False; 6. False; 7. True; 8. True; 9. False; 10. True.

Short Answer Questions

- State the probability integral transformation theorem.
- Define the quantile function using the infimum notation.
- Explain why U and $1-U$ can be used interchangeably in simplifying quantile functions.
- Why does the inverse transformation method require a search procedure in the discrete case but not in the continuous case?
- What is the compatibility relation, and why is it central to the proof of Theorem 3.2?
- Give one example of a distribution for which the quantile function has no elementary closed form.
- Explain the geometric interpretation of reading a horizontal line at height u across the graph of F_X .
- What is the time complexity of the discrete algorithm using a linear search, and how is it improved?
- Why is the inverse transformation method described as “exact”?
- Describe one application of the inverse transformation method in reliability engineering.

Long Answer Questions

1. Derive the probability integral transformation theorem, and explain its role in goodness-of-fit testing.
2. State and prove the inverse transformation theorem for a general cdf, without assuming continuity.
3. Derive the quantile function for the Exponential distribution and explain the simplification obtained using the identity between U and $1 - U$.
4. Derive the discrete inverse transformation formula for the Geometric distribution, showing every algebraic step.
5. Compare the computational complexity of the continuous and discrete inverse transformation algorithms, and explain the source of the difference.
6. Explain, with a worked numerical example, how the inverse transformation method is used for parametric bootstrap resampling.
7. Discuss the limitations of the inverse transformation method and describe at least one alternative technique appropriate when these limitations apply.
8. Explain how `findInterval()` implements the discrete inverse transformation algorithm, and compare it with a manual `for` loop implementation in terms of both correctness and speed.
9. Discuss the historical development of the inverse transformation method and its relationship to early Monte Carlo simulation.
10. Using the compatibility relation, prove that the quantile function is non-decreasing and left-continuous.

Numerical Problems

1. Given $f(x) = 2x$, $0 < x < 1$, derive $F_X^{-1}(u)$ and compute the simulated value for $u = 0.36$.
2. For $X \sim \text{Exponential}(\lambda = 3)$, compute the simulated value corresponding to $u = 0.1$.
3. For $X \sim \text{Bernoulli}(p = 0.7)$, determine the simulated values corresponding to $u = 0.2, 0.5, 0.9$.
4. For $X \sim \text{Geometric}(p = 0.3)$, compute the simulated value corresponding to $u = 0.4$.
5. Given the discrete support $\{1, 2, 3\}$ with probabilities $0.2, 0.5, 0.3$, compute the simulated value for $u = 0.65$.
6. For the density $f(x) = 4x^3$, $0 < x < 1$, derive $F_X^{-1}(u)$ and compute the value for $u = 0.81$.
7. For $X \sim \text{Uniform}(a, b)$, derive the general quantile function and simulate a value for $a = 2, b = 10, u = 0.25$.
8. For a two-point distribution with $P(X = 0) = 0.55$, $P(X = 1) = 0.45$, compute the generated value for $u = 0.5$ and $u = 0.6$.
9. For X with cdf $F_X(x) = 1 - e^{-x^2}$, $x > 0$ (Rayleigh-type), derive $F_X^{-1}(u)$ and compute the value for $u = 0.5$.
10. For $X \sim \text{Geometric}(p = 0.5)$, compute the simulated values corresponding to $u = 0.1, 0.3, 0.5, 0.7, 0.9$ and tabulate them.

Programming Exercises

1. Write an R function `qcubic_inverse(u)` that returns $F_X^{-1}(u) = u^{1/3}$ and use it to simulate $n = 1000$ observations from $f(x) = 3x^2$.
2. Write an R function to simulate n observations from $\text{Exponential}(\lambda)$ using only `runif()` and elementary arithmetic, and compare the output to `rexp()` using a Q-Q plot.
3. Implement the discrete inverse transformation algorithm from first principles (without `findInterval()`) using a `for` loop, and benchmark it against a vectorized `findInterval()` implementation using `system.time()`.
4. Write an R function `rgeom_custom(n, p)` implementing the inverse transformation algorithm for the Geometric distribution, and verify its output against `rgeom(n, p)` using summary statistics.
5. Write an R function that simulates from an arbitrary discrete distribution given a vector of support points and a vector of probabilities, validating that the probabilities sum to one.
6. Simulate $n = 1000$ $\text{Bernoulli}(p = 0.4)$ variates using inverse transformation and verify the sample proportion using a one-sample proportion test (`prop.test()`).
7. Write an R function implementing the quantile function for the $\text{Uniform}(a, b)$ distribution and use it to simulate a sample, confirming the sample range lies within (a, b) .

8. Using `ggplot2`, produce a histogram overlaid with the theoretical density for a distribution of your choice, generated via inverse transformation.
9. Write an R script that simulates the Logarithmic(θ) distribution using inverse transformation by constructing and searching a cumulative probability vector, extending the vector adaptively if the initial length is insufficient.
10. Write an R function that performs a Kolmogorov-Smirnov test (`ks.test()`) comparing a simulated sample obtained via inverse transformation to the theoretical target distribution, and report the p-value.

Mini Projects

1. **Simulation-based validation of custom generators.** Implement inverse transformation generators for three distributions not directly available in base R (for example, the Logarithmic distribution, a discrete triangular distribution, and a truncated Exponential distribution). For each, validate the generator using graphical comparison (density or bar plots) and a formal goodness-of-fit test, and summarize your findings in a short report.
2. **Parametric bootstrap via inverse transformation.** Fit a parametric distribution (e.g. Gamma or Weibull) to a real or simulated dataset, and implement a parametric bootstrap procedure that resamples from the fitted distribution using the inverse transformation method rather than `rgamma()` or `rweibull()` directly. Compare bootstrap confidence intervals for the mean obtained from your custom generator against those obtained from the built-in generator, and comment on any discrepancies.

Difficulty labels. Multiple choice, fill in the blanks, true or false: **Easy**. Short answer, numerical problems 1–6: **Medium**. Long answer, numerical problems 7–10, programming exercises: **Medium to Advanced**. Mini projects: **Advanced**.

References

- Devroye, L. (1986). *Non-uniform random variate generation*. Springer-Verlag.
- Fishman, G. S. (1996). *Monte Carlo: Concepts, algorithms, and applications*. Springer-Verlag.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.